Fundamental Vector Commitment Study: Extended Pedersen vs Catalano-Fiore

Tanggal: 20 September 2026. Status: riset teoretis dan pseudocode primitive. Tidak ada implementasi Rust, benchmark, membership proof, atau desain payment dalam dokumen ini. Istilah **EP** mengikuti rumus Pedersen-style vector commitment pada PDF deskripsi proyek, bukan nama skema dengan selective opening yang sudah ditetapkan secara terpisah.

1. Pertanyaan yang dinilai

Untuk vektor skalar terurut `v=(v_1,...,v_n)`, primitive mana yang lebih cocok sebagai **lapisan commitment**: rumus Extended Pedersen (EP) dari PDF proyek, atau keluarga Catalano-Fiore (CF)? CF mempunyai dua instansiasi berbeda: CF-CDH dan CF-RSA. `n` berarti **jumlah posisi dalam satu vektor**, bukan jumlah run benchmark dan bukan otomatis jumlah commitment terpisah.

Perbandingan yang adil harus membedakan:

1. `Commit`: mengikat satu vektor.
2. `Open(i)`/`Verify(i)`: membuka **satu posisi** tanpa membuka seluruh vektor.
3. `Update(i)`/`ProofUpdate(i)`: mengganti satu posisi dan menyegarkan opening.
4. Hiding, position binding, panjang parameter publik, dan asumsi setup.

Definisi CF meminta commitment dan opening per-posisi berukuran independen dari `n`; position binding berarti satu commitment tidak dapat dibuka ke dua nilai berbeda pada posisi yang sama. Rumus EP sendiri adalah *vector-wide Pedersen commitment*: commitment-nya pendek, tetapi **belum menyediakan** `Open(i)` **ringkas**. Karena itu, menyebut keduanya "VC" tidak berarti API dan properti yang tersedia sudah setara. Sumber: CF, Section 2.

Notasi bersama: `lambda` = parameter keamanan; `M` = ruang pesan skalar; `[n]={1,...,n}`; `C` = commitment; `aux` = state rahasia committer; `Lambda_i` = opening posisi `i`. Semua rumus di bawah menggunakan notasi grup **multiplikatif**, sesuai PDF proyek dan paper CF. Seluruh nilai dan indeks harus berada pada domain yang dinyatakan dalam parameter.

2. Protocol 1: Extended Pedersen Vector-Wide Commitment

Parameter: grup siklik `G` berorde prima `q`, `h,g_1,...,g_n in G`, generator yang relasi discrete-log antar-basisnya tidak diketahui pihak mana pun. `pp_EP=(G,q,h,g_1,...,g_n)`, `M=Z_q`. Derivasi basis dengan hash-to-group ber-domain-terpisah adalah opsi engineering, bukan bukti keamanan dengan sendirinya.

Input: `Setup(lambda,n)`; `Commit(pp_EP,v)` untuk `v in M^n`; `VerifyAll(pp_EP,C,v,r)`; `Update(pp_EP,C,i,v_i,v_i')`.

Output: `Commit -> (C,aux=(v,r))`; `VerifyAll -> ACCEPT/ABORT`; `Update -> C'`. Interface `OpenAt(i)` yang ringkas **tidak tersedia dari rumus ini saja**.

```
Protocol EP-1 (Committer / Verifier)

Setup(lambda,n):
  1. Fix a prime-order group G of order q.
  2. Establish h,g_1,...,g_n in G with unknown log relations.
  3. Publish pp_EP = (G,q,h,g_1,...,g_n).

Committer.Commit(pp_EP, v=(v_1,...,v_n)):
  4. Check n positions and v_i in Z_q for every i.
  5. Sample r uniformly from Z_q.
  6. Compute C = h^r * product_{i=1}^n g_i^(v_i).
  7. Return (C, aux=(v,r)).

Committer.OpenAll(aux):
  8. Return (v,r).  // Discloses all n messages.

Verifier.VerifyAll(pp_EP,C,v,r):
  9. Validate C in G and every v_i,r in Z_q.
 10. ACCEPT iff C == h^r * product_{i=1}^n g_i^(v_i).

Committer.Update(C,i,v_i,v_i',aux):
 11. Let Delta = v_i' - v_i mod q.
 12. Set C' = C * g_i^Delta; replace aux.v_i by v_i'.
 13. Return (C',aux').

OpenAt(i): NOT DEFINED by this bare construction.
ProofUpdate(i): NOT DEFINED without an added positional proof system.
```

Correctness: `C' = h^r * product_{j != i} g_j^(v_j) * g_i^(v_i')`. Random `r` membuat `C` terdistribusi uniform dalam `G` untuk setiap vektor, sehingga hiding vektor penuh sempurna. Binding bergantung pada sulitnya mencari relasi discrete-log nontrivial antar-generator; tidak ada klaim selective opening anonim dari rumus ini. Ukuran `C`: satu elemen `G`; parameter publik: `n+1` elemen `G`; full opening: `n+1` skalar.

3. Protocol 2: Catalano-Fiore CDH Vector Commitment

Parameter: bilinear groups `G,G_T` berorde prima `p`, pairing `e:G x G -> G_T`, generator `g in G`; `M=Z_p`. KeyGen memilih `z_i in Z_p`, lalu `h_i=g^(z_i)` dan `h_{i,j}=g^(z_i*z_j)` untuk `i != j`. `pp_CDH=(G,G_T,p,e,g,{h_i},{h_{i,j}})`. Nilai `z_i` adalah rahasia setup; penggunaan setup aman/erasure adalah pertimbangan deployment yang tidak dibuktikan oleh pseudocode ini.

Input: `KeyGen(lambda,n)`; `Commit(pp_CDH,m)`; `Open(pp_CDH,m,i)`; `Verify(pp_CDH,C,m_i,i,Lambda_i)`; `Update(C,t,m_t,m_t')` dan `ProofUpdate(Lambda_i,t,m_t,m_t')`.

Output: `Commit -> (C,aux=m)`; `Open -> Lambda_i` (satu elemen `G`); `Verify -> ACCEPT/ABORT`; `Update -> (C',U)`; `ProofUpdate -> Lambda_i'`.

```
Protocol CF-CDH-1 (Setup / Committer / Verifier)

Setup.KeyGen(lambda,n):
  1. Generate bilinear groups G,G_T of prime order p and e:G x G -> G_T.
  2. Choose generator g in G and secret z_1,...,z_n in Z_p.
  3. Set h_i = g^(z_i); h_{i,j} = g^(z_i*z_j), i != j.
  4. Publish pp_CDH; protect/erase all z_i.

Committer.Commit(pp_CDH,m=(m_1,...,m_n)):
  5. Check all m_i in Z_p.
  6. Compute C = product_{i=1}^n h_i^(m_i).
  7. Return (C,aux=m).

Committer.Open(pp_CDH,aux=m,i):
  8. Compute Lambda_i = product_{j != i} h_{i,j}^(m_j).
  9. Return Lambda_i.

Verifier.Verify(pp_CDH,C,m_i,i,Lambda_i):
 10. Validate i, m_i and group elements.
 11. ACCEPT iff e(C / h_i^(m_i), h_i) == e(Lambda_i,g).

Committer.Update(C,t,m_t,m_t'):
 12. Delta = m_t' - m_t mod p; C' = C * h_t^Delta.
 13. Return (C',U=(t,m_t,m_t')).

Holder.ProofUpdate(Lambda_i,U):
 14. If i == t, return Lambda_i.
 15. Otherwise return Lambda_i' = Lambda_i * h_{i,t}^Delta.
```

Correctness: `e(C/h_i^(m_i),h_i) = e(product_{j != i}h_j^(m_j),g^(z_i)) = e(product_{j != i}h_{i,j}^(m_j),g)`. Paper membuktikan position binding berdasar Square-CDH/CDH, tetapi skema dasarnya deterministik dan **tidak hiding**. Commitment dan opening masing-masing satu elemen `G`; parameter yang dibutuhkan committer `O(n^2)` elemen grup, sedangkan verifier tidak membutuhkan seluruh `h_{i,j}`. Opening publik mengungkap posisi `i` dan nilai `m_i`. Sumber: CF, Sections 2 dan 2.1.

4. Protocol 3: Catalano-Fiore RSA Vector Commitment

Parameter: modulus RSA `N=p_1*p_2`, elemen publik `a in Z_N^*`, prima berbeda `e_1,...,e_n` sepanjang `ell+1` bit dengan `gcd(e_i,phi(N))=1`, dan `S_i = a^(product_{k != i} e_k) mod N`. Ruang pesan `M={0,1}^ell`, ditafsir sebagai integer. `pp_RSA=(N,a,{S_i},{e_i})`. Faktorisasi `N` harus tidak tersedia pada pembukti yang tidak dipercaya. Verifier perlu memastikan parameter publik valid sesuai KeyGen; paper menyebut pemeriksaan validitas key sekali di awal.

Input: `KeyGen(lambda,ell,n)`; `Commit(pp_RSA,m)`; `Open(pp_RSA,m,i)`; `Verify(pp_RSA,C,m_i,i,Lambda_i)`; `Update(C,t,m_t,m_t')`; `ProofUpdate(Lambda_i,t,m_t,m_t')`.

Output: `Commit -> (C,aux=m)`; `Open -> Lambda_i` (satu residu mod `N`); `Verify -> ACCEPT/ABORT`; `Update -> (C',U)`; `ProofUpdate -> Lambda_i'`.

```
Protocol CF-RSA-1 (Setup / Committer / Verifier)

Setup.KeyGen(lambda,ell,n):
  1. Generate N=p_1*p_2 and public a in Z_N^*.
  2. Choose distinct (ell+1)-bit primes e_i with gcd(e_i,phi(N))=1.
  3. Compute S_i = a^(product_{k != i} e_k) mod N.
  4. Publish pp_RSA; protect/erase the factorization of N.

Committer.Commit(pp_RSA,m=(m_1,...,m_n)):
  5. Check m_i in {0,1}^ell for all i.
  6. Compute C = product_{i=1}^n S_i^(m_i) mod N.
  7. Return (C,aux=m).

Committer.Open(pp_RSA,aux=m,i):
  8. For each j != i, compute B_{i,j} =
       a^(product_{k notin {i,j}} e_k) mod N using public data.
  9. Compute Lambda_i = product_{j != i} B_{i,j}^(m_j) mod N.
 10. Return Lambda_i.  // No factorization needed.

Verifier.Verify(pp_RSA,C,m_i,i,Lambda_i):
 11. Check i, m_i in M, and valid public parameters.
 12. ACCEPT iff C == S_i^(m_i) * Lambda_i^(e_i) mod N.

Committer.Update(C,t,m_t,m_t'):
 13. Delta = m_t' - m_t as an integer; C' = C * S_t^Delta mod N.
 14. Return (C',U=(t,m_t,m_t')).

Holder.ProofUpdate(Lambda_i,U):
 15. If i == t, return Lambda_i.
 16. Otherwise return Lambda_i' = Lambda_i * B_{i,t}^Delta mod N.
```

Untuk `Delta < 0`, pangkat negatif berarti invers grup; hanya elemen unit valid yang boleh dipakai. `B_{i,j}` diturunkan dari `a` dan prima publik; persamaan `B_{i,j}^(e_i)=S_j` menunjukkan kebenaran opening dan proof update. Skema dasar deterministik dan **tidak hiding**. Commitment dan opening masing-masing satu residu mod `N`; parameter publik dasar `O(n)` elemen/eksponen, tetapi perhitungan basis dan eksponennya tidak boleh disamakan dengan biaya konstan per operasi tanpa parameter konkret. Opening publik mengungkap `i` dan `m_i`. Sumber: CF, Section 2.2.

5. Matriks perbandingan primitive

Aspek	EP (rumus PDF)	CF-CDH	CF-RSA
Komitmen	1 elemen grup prime-order	1 elemen pairing group `G`	1 residu mod `N`
`Open(i)`	Tidak ada dalam rumus dasar; full opening `n+1` skalar	1 elemen `G`	1 residu mod `N`
`Verify(i)`	Butuh protokol tambahan; full opening `O(n)`	Persamaan pairing	Persamaan modular
Hiding bawaan	Sempurna untuk vektor sebelum opening	Tidak	Tidak
Binding yang dibandingkan	Vektor penuh, asumsi relasi DL tidak diketahui	Position binding, Square-CDH/CDH	Position binding, RSA
Setup/parameter dasar	`O(n)` generator; derivasi transparan memungkinkan	`O(n^2)` parameter committer; bilinear setup	`O(n)` elemen/eksponen; modulus RSA
Update satu posisi	Rumus `C*g_i^Delta`; tidak ada proof update bawaan	Rumus `C*h_i^Delta` dan satu proof update	Rumus `C*S_i^Delta` dan satu proof update
Privasi indeks pada opening biasa	Tidak tersedia	Indeks publik	Indeks publik
Kecocokan stack saat ini	Satu keluarga dengan Ristretto/Pedersen	Memerlukan pairing library	Memerlukan RSA/big-int stack

Semua ukuran di atas adalah **jumlah elemen**, bukan byte benchmark. Satu elemen Ristretto, elemen pairing, dan residu RSA tidak boleh diasumsikan memiliki ukuran byte atau biaya operasi yang sama. Kompleksitas `O(n)` menyatakan pertumbuhan teoretis, bukan waktu milidetik terukur. Update satu opening tidak berarti memperbarui semua `n` opening dengan biaya konstan.

6. Kesimpulan pemilihan fundamental

Jika prioritasnya commitment vektor yang hiding dan kompatibel dengan aljabar Pedersen pada PDF, EP adalah kandidat awal. Rumusnya sederhana dan tidak memerlukan pairing/RSA, tetapi belum memenuhi definisi *concise VC* CF karena tidak ada opening posisi ringkas bawaan. Ini cocok untuk meneliti vektor tersembunyi, bukan alasan untuk mengklaim proof membership sudah jadi.

Jika prioritasnya verifiable opening posisi publik yang ringkas dan updatable, CF lebih tepat. CF-CDH memberi opening elemen grup dengan pairing dan parameter committer kuadratik; CF-RSA memberi opening residu modulus RSA dan setup modulus. Keduanya tidak hiding bawaan; privacy layer tambahan akan mengubah biaya dan relasi yang dinilai. Di antara CF-CDH dan CF-RSA, dokumen ini **belum memilih pemenang** tanpa parameter keamanan, model setup, dan implementasi konkret.

Untuk rumusan PDF proyek `v_1=x` setelah permutasi rahasia, manfaat native CF berupa opening posisi arbitrer yang ringkas tidak otomatis terpakai. Sebaliknya, EP yang hiding belum membuktikan bahwa vektor itu permutasi sah. **Pilihan primitive dan rancangan proof membership adalah dua keputusan berbeda.** Pertanyaan tentang root tepercaya, pemilik `c_tid`, dan server payment sengaja tidak diselesaikan di tahap pemilihan primitive ini.

7. Asal klaim

- Project Description PDF, halaman 1: rumus Pedersen-style `C=h^r product g_i^(v_i)` dan motivasi vektor tersembunyi. Rumus EP/Update/correctness di dokumen ini adalah penjabaran aljabar dari rumus tersebut, **bukan** algoritme opening posisi dari PDF.
- Catalano dan Fiore, *Vector Commitments and their Applications*, PKC 2013, Sections 2, 2.1, 2.2: definisi VC, CDH, RSA, update, posisi binding, hiding yang tidak diwajibkan.
- Kajian pendukung sebelumnya: konteks riset aplikasi set membership. Dokumen pendukung itu lebih luas dari scope pemilihan primitive ini.